

Name - Chaitanya Kelkar

Roll no - 15

Experiment No 2

Theory

1. What is a Blockchain?

A blockchain is a distributed, decentralized ledger that stores data in a chain of linked blocks. Each block holds transactions, a timestamp, a proof (nonce), and the hash of the previous block. Since every block references the hash of the one before it, altering any past block breaks the chain of hashes that follow — making the ledger tamper-evident and effectively immutable.

2. Block Structure

Each block in this implementation is a dictionary with: **index** (position in chain), **timestamp**, **proof** (PoW result), **previous_hash**, and **transactions** (the batch of transactions included in that block).

3. Pending Transactions

New transactions are not written directly to the chain. `add_transaction()` first adds them to a temporary pending list. This models real blockchains, where submitted transactions wait in a pool until a miner bundles them into a block. Once mined, the pending list is embedded in the new block and then cleared.

4. Proof-of-Work (Mining)

Mining means solving a computational puzzle to earn the right to add a block. `proof_of_work()` searches for a nonce such that $\text{SHA256}(\text{new_proof}^2 - \text{previous_proof}^2)$ produces a hash starting with "000" (3 leading zeros) — the **golden nonce**. This search is brute-force and unpredictable by design, making block creation intentionally slow and resource-intensive, which secures the chain against easy tampering. Mining is only permitted when at least one transaction is pending, avoiding wasted effort on empty blocks.

5. Hashing and Immutability

`hash()` serializes a block (JSON, sorted keys) and applies SHA-256 to get a fixed-length hex digest. SHA-256 is one-way, deterministic, and highly sensitive to change (avalanche effect) — even a single altered character produces a completely different hash. This hash becomes the `previous_hash` of the next block, cryptographically linking blocks together.

6. Chain Validation

`is_chain_valid()` checks every block against two conditions: its `previous_hash` must match the actual hash of the prior block, and its proof must still satisfy the "000" PoW condition

when recomputed. Any mismatch anywhere in the chain marks it invalid, ensuring tampering is detectable.

7. Flask API Layer

The logic is exposed via Flask routes — `/add_transaction`, `/mine_block`, `/get_chain`, and `/is_valid` — allowing transactions, mining, and validation to be triggered through simple web requests, similar to how real blockchain nodes expose RPC endpoints.

Output -

Code showing Blockchain class core methods (`__init__`, `create_block()`, `get_previous_block()`, `add_transaction()`)

```
import datetime
import hashlib
import json
from flask import Flask, jsonify, request

class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = [] # Pending transactions list
        # Genesis block
        self.create_block(proof=1, previous_hash='0')

    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': self.transactions # Include pending transactions
        }
        self.transactions = [] # Reset/clear pending transactions list
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def add_transaction(self, sender, receiver, amount):
        """Adds a new transaction to the pending transactions list."""
        self.transactions.append({
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        })
        previous_block = self.get_previous_block()
        return previous_block['index'] + 1
```

Code showing proof_of_work() (3 leading zeros), hash(), is_chain_valid(), Flask app init, and /add_transaction route

```
def proof_of_work(self, previous_proof):
    """Searches for golden nonce matching 3 leading zeros ('000')."""
    new_proof = 1
    check_proof = False
    while not check_proof:
        hash_operation = hashlib.sha256(
            str(new_proof**2 - previous_proof**2).encode()
        ).hexdigest()
        # Cryptographic Puzzle: 3 leading zeros
        if hash_operation[:3] == '000':
            check_proof = True
        else:
            new_proof += 1
    return new_proof

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(
            str(proof**2 - previous_proof**2).encode()
        ).hexdigest()
        # Validate 3 leading zeros
        if hash_operation[:3] != '000':
            return False
        previous_block = block
        block_index += 1
    return True
```

Code showing /mine_block, /get_chain, and /is_valid Flask routes

```
# Initialize Flask app
app = Flask(__name__)
blockchain = Blockchain()
```

```
@app.route('/add_transaction', methods=['POST'])
def add_transaction():
    json_data = request.get_json()
    transaction_keys = ['sender', 'receiver', 'amount']
    if not all(key in json_data for key in transaction_keys):
        return 'Some elements of the transaction are missing', 400
    index = blockchain.add_transaction(json_data['sender'], json_data['receiver'], json_data['amount'])
    response = {'message': f'Transaction will be added to Block {index}'}
    return jsonify(response), 201
```

```
@app.route('/mine_block', methods=['GET'])
def mine_block():
    # Mine only if pending transactions are present
    if not blockchain.transactions:
        return jsonify({'message': 'Cannot mine a block: Transaction list is empty!'}), 400

    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)

    block = blockchain.create_block(proof, previous_hash)
    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }
    return jsonify(response), 200
```

```
@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {'chain': blockchain.chain, 'length': len(blockchain.chain)}
    return jsonify(response), 200
```

```
@app.route('/is_valid', methods=['GET'])
def is_valid():
    valid = blockchain.is_chain_valid(blockchain.chain)
    if valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'The Blockchain is not valid.'}
    return jsonify(response), 200
```

```

print("Functions Menu:")
print("=====")
print("1. Add a transaction")
print("2. Mine a block")
print("3. Display the chain")
print("4. Check the validity of the chain")

try:
    choice = int(input("Enter your choice: "))

    # Wrap calls in Flask request context to support jsonify() directly
    with app.test_request_context():
        if choice == 1:
            sender = input("Enter Sender: ") or "Alice"
            receiver = input("Enter Receiver: ") or "Bob"
            amount = float(input("Enter Amount: ") or 10)
            idx = blockchain.add_transaction(sender, receiver, amount)
            print(f"Success: Transaction added to pending list for Block {idx}")
        elif choice == 2:
            res, status_code = mine_block()
            print(res.get_json())
        elif choice == 3:
            res, status_code = get_chain()
            print(res.get_json())
        elif choice == 4:
            res, status_code = is_valid()
            print(res.get_json())
        else:
            print("Invalid choice")
except ValueError:
    print("Invalid input. Please enter a valid number.")

```

Console output — Option 1 (Add Transaction: Chaitanya → Tejas, 12) followed by Option 2 (Mine a Block, showing mined block details)

```

Functions Menu:
=====
1. Add a transaction
2. Mine a block
3. Display the chain
4. Check the validity of the chain
Enter your choice: 1
Enter Sender: Chaitanya
Enter Receiver: Tejas
Enter Amount: 12
Success: Transaction added to pending list for Block 2

```

```

Functions Menu:
=====
1. Add a transaction
2. Mine a block
3. Display the chain
4. Check the validity of the chain
Enter your choice: 2
{'index': 2, 'message': 'Congratulations, you just mined a block!', 'previous_hash': '14f51b7b28b6994c41b0840d694967a6dcb45d8a9d1b01ba1fc8ac2824dee0b7',
'proof': 533, 'timestamp': '2026-09-08 17:55:31.800325', 'transactions': [{'amount': 15.0, 'receiver': 'Tejas', 'sender': 'Chaitanya'}]}

```

Console output — Option 3 (Display the Chain) followed by Option 4 (Check Validity: "All good. The Blockchain is valid.")

```
Functions Menu:
=====
1. Add a transaction
2. Mine a block
3. Display the chain
4. Check the validity of the chain
Enter your choice: 3
{'chain': [{'index': 1, 'previous_hash': '0', 'proof': 1, 'timestamp': '2026-09-08 19:30:17.614773', 'transactions': []}, {'index': 2, 'previous_hash': '79f1c5937f23731b4dc28
```

- Functions Menu:
=====
1. Add a transaction
2. Mine a block
3. Display the chain
4. Check the validity of the chain
Enter your choice: 4
{'message': 'All good. The Blockchain is valid.'}

Conclusion -

We successfully extended the base blockchain program to support pending transactions, conditional mining (only when transactions exist), and a modified Proof-of-Work requiring 3 leading zeros in the hash. All four operations — adding a transaction, mining a block, displaying the chain, and checking validity — were tested through the Functions Menu and produced correct, consistent results. This confirms the implementation upholds core blockchain principles of immutability, consensus, and transaction integrity in a simplified environment.